LOW LEVEL DESIGN

Banking Business Rules Engine (BRE)

Database Schema | Service Classes | API Contracts | Kafka Topics | Cache Strategy

Document	Low Level Design — BRE Platform
Version	1.0
Date	June 2026
Stack	Java 17 | Spring Boot 3.x | PostgreSQL 15 | Redis 7 | Kafka 3.x

1. Database Schema Design

1.1 Schema: bre_engine

All BRE tables reside in the bre_engine schema in PostgreSQL. Rule definitions, versions, execution logs, and audit data are separated into distinct tables for clean access patterns.

Table: bre_rule_definition

```
CREATE TABLE schema_bre.bre_rule_definition (     id                  BIGSERIAL
PRIMARY KEY,     rule_code           VARCHAR(50)  NOT NULL UNIQUE,     rule_name
VARCHAR(200) NOT NULL,     rule_category       VARCHAR(100) NOT NULL,    --
ELIGIBILITY | DEVIATION | COMPLIANCE | FRAUD     product_type        VARCHAR(50)
NOT NULL,    -- MFI | RETAIL | SME | ALL     applicability_stage VARCHAR(100) NOT
NULL,    -- PRE_SANCTION | POST_SANCTION | DISBURSEMENT     description
TEXT,     is_deleted          BOOLEAN      NOT NULL DEFAULT FALSE,     created_by
VARCHAR(50)  NOT NULL,     created_on          TIMESTAMP    NOT NULL DEFAULT
NOW(),     updated_by          VARCHAR(50),     updated_on          TIMESTAMP );
```

Table: bre_rule_version

```
CREATE TABLE schema_bre.bre_rule_version (     id                  BIGSERIAL
PRIMARY KEY,     rule_definition_id  BIGINT       NOT NULL REFERENCES
bre_rule_definition(id),     version_number      INT          NOT NULL,     status
VARCHAR(30)  NOT NULL DEFAULT 'DRAFT',                          -- DRAFT |
IN_REVIEW | APPROVED | ACTIVE | SUPERSEDED | DEACTIVATED     condition_dsl
JSONB        NOT NULL,    -- Rule logic as structured JSON     outcome_config
JSONB        NOT NULL,    -- PASS/DEVIATION/FAIL config     deviation_role
VARCHAR(100),             -- CU_L3 | CU_L4 | CREDIT_HEAD     effective_from
DATE,     effective_to        DATE,     approved_by         VARCHAR(50),
approved_on         TIMESTAMP,     activated_by        VARCHAR(50),
activated_on        TIMESTAMP,     created_by          VARCHAR(50)  NOT NULL,
created_on          TIMESTAMP    NOT NULL DEFAULT NOW(),     UNIQUE
(rule_definition_id, version_number) );
```

Table: bre_rule_execution_log

```
CREATE TABLE schema_bre.bre_rule_execution_log (     id                  BIGSERIAL
PRIMARY KEY,     execution_group_id  UUID         NOT NULL,    -- Groups all rules
evaluated for one request     loan_app_id         BIGINT,     entity_id
BIGINT       NOT NULL,     entity_type         VARCHAR(50)  NOT NULL,    --
LOAN_APP | BORROWER | GROUP     rule_version_id     BIGINT       NOT NULL
REFERENCES bre_rule_version(id),     rule_code           VARCHAR(50)  NOT NULL,
rule_status         SMALLINT     NOT NULL,    -- 1=PASS 2=DEVIATION 3=FAIL
note_code           VARCHAR(100),     note                TEXT,
deviated_to_role    VARCHAR(100),     input_snapshot      JSONB,
-- Masked PII copy of evaluation inputs     result              JSONB,
-- Full condition evaluation trace     evaluation_ms       INT,
-- Time taken in milliseconds     triggered_by        VARCHAR(100) NOT NULL,    --
system_user or service name     created_on          TIMESTAMP    NOT NULL DEFAULT
NOW() ); CREATE INDEX idx_exec_log_loan_app   ON
schema_bre.bre_rule_execution_log(loan_app_id); CREATE INDEX
idx_exec_log_rule_code  ON schema_bre.bre_rule_execution_log(rule_code); CREATE
INDEX idx_exec_log_created_on ON schema_bre.bre_rule_execution_log(created_on
DESC);
```

Table: bre_condition_dsl — JSONB Structure

```
-- condition_dsl JSONB example for SHG0003 (VRM Category Check) {   "operator":
"AND",   "conditions": [     {       "field": "vrm_base_category",
"operator": "IN",       "value": ["A", "B"],       "outcome": "PASS"     },     {
"field": "vrm_base_category",       "operator": "EQ",       "value": "C",
"outcome": "DEVIATION",       "deviation_role": "CU_L3"     },
{       "field": "vrm_base_category",       "operator": "IN",       "value": ["D",
"E"],       "outcome": "FAIL"     }   ] }
```

2. Service Layer — Class Design

2.1 RuleExecutionService

```
@Service @Slf4j public class RuleExecutionService {      private final
RuleCacheService       ruleCacheService;     private final RuleEvaluator
ruleEvaluator;     private final DeviationAggregator    deviationAggregator;
private final AuditLogService        auditLogService;     private final
BREEventPublisher      eventPublisher;      public EvaluationResponse
evaluate(EvaluationRequest request) {         String executionGroupId =
UUID.randomUUID().toString();         List<RuleVersion> rules =
ruleCacheService.getActiveRules(             request.getProductType(),
request.getRuleGroup());          List<RuleResult> results = rules.stream()
.map(rule -> ruleEvaluator.evaluate(rule,
request.getContext()))             .collect(Collectors.toList());
DeviationSummary deviation = deviationAggregator.aggregate(results);
auditLogService.persist(executionGroupId, request, results);
eventPublisher.publishEvaluationCompleted(executionGroupId, results);
return
EvaluationResponse.builder()             .executionGroupId(executionGroupId)
.overallStatus(resolveOverallStatus(results))             .ruleResults(results)
.deviationSummary(deviation)             .build();     }      private RuleStatus
resolveOverallStatus(List<RuleResult> results) {         if
(results.stream().anyMatch(r -> r.getStatus() == RuleStatus.FAIL))
return RuleStatus.FAIL;         if (results.stream().anyMatch(r -> r.getStatus()
== RuleStatus.DEVIATION))             return RuleStatus.DEVIATION;         return
RuleStatus.PASS;     } }
```

2.2 RuleEvaluator

```
@Component public class RuleEvaluator {      private final ExpressionParser spel =
new SpelExpressionParser();      public RuleResult evaluate(RuleVersion
ruleVersion, Map<String, Object> context) {         long start =
System.currentTimeMillis();         ConditionDSL dsl =
ruleVersion.getConditionDsl();         RuleOutcome outcome =
evaluateConditionTree(dsl, context);          return RuleResult.builder()
.ruleCode(ruleVersion.getRuleCode())             .ruleVersionId(ruleVersion.getId(
))             .status(outcome.getStatus())             .noteCode(outcome.getNoteC
ode())             .note(outcome.getNote())             .deviatedToRole(outcome.ge
tDeviationRole())             .evaluationMs((int)(System.currentTimeMillis() -
start))             .build();     }      private RuleOutcome
evaluateConditionTree(ConditionDSL node, Map<String, Object> ctx) {         if
(node.isLeaf()) {             return evaluateSingleCondition(node, ctx);         }
```

```
// AND: all must pass; OR: any pass is sufficient        List<RuleOutcome>
childOutcomes = node.getConditions().stream()            .map(child ->
evaluateConditionTree(child, ctx))            .collect(Collectors.toList());
return "AND".equals(node.getOperator())            ? resolveAnd(childOutcomes)
: resolveOr(childOutcomes);     }      private RuleOutcome
evaluateSingleCondition(ConditionDSL cond, Map<String, Object> ctx)
{         StandardEvaluationContext spelCtx = new StandardEvaluationContext();
spelCtx.setVariables(ctx);         String expr = buildSpelExpression(cond);
boolean result = Boolean.TRUE.equals(spel.parseExpression(expr).getValue(spelCtx,
Boolean.class));         return result ? RuleOutcome.pass() :
RuleOutcome.fromCondition(cond);     } }
```

2.3 RuleCacheService

```
@Service public class RuleCacheService {      private static final String
CACHE_KEY_PREFIX = "bre:rules:active:";     private static final Duration
CACHE_TTL = Duration.ofMinutes(30);      private final StringRedisTemplate redis;
private final RuleVersionRepository ruleVersionRepository;     private final
ObjectMapper objectMapper;      public List<RuleVersion> getActiveRules(String
productType, String ruleGroup) {         String key = CACHE_KEY_PREFIX +
productType + ":" + ruleGroup;         try {            String cached =
redis.opsForValue().get(key);            if (cached != null)
{                return objectMapper.readValue(cached,                  new
TypeReference<List<RuleVersion>>() {});            }        } catch (Exception
e) {            log.warn("Redis cache miss for key: {}, falling back to DB",
key);         }         // DB fallback         List<RuleVersion> rules =
ruleVersionRepository             .findActiveByProductTypeAndRuleGroup(productType
, ruleGroup);         try {            redis.opsForValue().set(key,
objectMapper.writeValueAsString(rules), CACHE_TTL);         } catch (Exception e)
{            log.warn("Failed to cache rules for key: {}", key);         }
return rules;     }      public void invalidateCache(String productType, String
ruleGroup) {         redis.delete(CACHE_KEY_PREFIX + productType + ":" +
ruleGroup);         log.info("[CACHE] Invalidated BRE rule cache for {}:{}",
productType, ruleGroup);     } }
```

2.4 DeviationAggregator

```
@Component public class DeviationAggregator {      // Role hierarchy — higher
index = higher authority     private static final List<String> ROLE_HIERARCHY =
List.of("CU_L3", "CU_L4", "CREDIT_HEAD", "MD");      public DeviationSummary
aggregate(List<RuleResult> results) {         List<RuleResult> deviations =
results.stream()             .filter(r -> r.getStatus() == RuleStatus.DEVIATION)
.collect(Collectors.toList());          if (deviations.isEmpty()) return
DeviationSummary.none();          String highestRole = deviations.stream()
.map(RuleResult::getDeviatedToRole)             .filter(Objects::nonNull)
.max(Comparator.comparingInt(ROLE_HIERARCHY::indexOf))             .orElse("CU_L3"
);          return DeviationSummary.builder()             .hasDeviation(true)
.escalationRole(highestRole)             .deviatedRules(deviations.stream().map(Ru
leResult::getRuleCode).collect(Collectors.toList()))             .deviationCount(d
eviations.size())             .build();     } }
```

3. API Contracts

3.1 POST /api/v1/bre/evaluate — Request

```
{   "loanAppId"   : ENTITY_ID,   "entityType"  : "LOAN_APP",   "productType" :
"MFI",   "ruleGroup"   : "PRE_SANCTION",   "triggeredBy" : "LOS_SERVICE",
"context": {     "vrm_base_category"   : "C",     "vrm_risk_category"   : "C2",
"group_cycle_type"    : "SHG_INITIAL",     "group_size"          : 12,     "foir"
: 0.42,     "cibil_score"         : 710,     "monthly_income"      : 18000,
"total_emi"           : 4200,     "active_loan_count"   : 1,     "loan_amount"
: 50000,     "loan_product_code"   : "MFI_GLP_001"   } }
```

3.2 POST /api/v1/bre/evaluate — Response

```
{   "executionGroupId" : "f3a2b1c0-1234-5678-abcd-ef0123456789",   "loanAppId"
: ENTITY_ID,   "overallStatus"    : "DEVIATION",   "evaluatedAt"      : "2026-06-
09T10:30:00.123Z",   "totalRules"       : 5,   "passCount"        : 3,
"deviationCount"   : 1,   "failCount"        : 0,   "deviationSummary" :
{     "hasDeviation"   : true,     "escalationRole" : "CU_L3",
"deviationCount" : 1,     "deviatedRules"  : ["SHG0003"]   },   "ruleResults": [
{       "ruleCode"       : "SHG0003",       "ruleVersionId"  : 42,       "status"
: "DEVIATION",       "statusLabel"    : "DEVIATION",       "noteCode"       :
"VRM_C_DEVIATION",       "note"           : "Village risk category C requires L3
approval",       "deviatedToRole" : "CU_L3",       "evaluationMs"   : 12     },
{       "ruleCode"       : "MFI0001",       "ruleVersionId"  : 18,       "status"
: "PASS",       "statusLabel"    : "PASS",       "noteCode"       : null,
"note"           : null,       "deviatedToRole" : null,       "evaluationMs"   : 8
}   ] }
```

4. Kafka Topics & Event Contracts

Topic Name	Producer	Consumer	Payload
bre.evaluation.request	LOS	BRE Service	EvaluationRequest JSON
bre.evaluation.completed	BRE Service	LOS, Reporting	EvaluationResponse JSON
bre.deviation.raised	BRE Service	Notification Svc	DeviationEvent: loanAppId, role, rules
bre.rule.activated	BRE Service	BRE Service (self)	RuleActivatedEvent: ruleCode, version — triggers cache invalidation
bre.rule.deactivated	BRE Service	BRE Service (self)	RuleDeactivatedEvent: ruleCode — triggers cache invalidation

4.1 Kafka Consumer Configuration

```
@KafkaListener(     topics = "bre.evaluation.request",     groupId = "bre-async-
```

```
evaluator",     containerFactory = "breKafkaListenerContainerFactory" ) public
void handleAsyncEvaluationRequest(         ConsumerRecord<String, String> record,
Acknowledgment ack) {     try {         EvaluationRequest request =
objectMapper.readValue(             record.value(), EvaluationRequest.class);
EvaluationResponse response = ruleExecutionService.evaluate(request);
eventPublisher.publishEvaluationCompleted(response);         ack.acknowledge();
} catch (Exception e) {         log.error("[BRE-CONSUMER] Failed to process
evaluation request. "             + "Topic: {}, Offset: {}, Error: {}",
record.topic(), record.offset(), e.getMessage(), e);         // Do NOT ack — will
be reprocessed or DLQ     } }
```

5. Redis Cache Strategy

Cache Key Pattern	TTL	Invalidation Trigger	Notes
bre:rules:active:{product}:{group}	30 minutes	Rule activated/deactivated event	Preloaded on first request
bre:bureau:cibil:{pan}	24 hours	Manual flush or scheduled job	Bureau API is expensive — cache aggressively
bre:vrm:{vtc_id}	6 hours	VRM data update event	Village risk rarely changes intraday
bre:blacklist:{pan}	1 hour	Blacklist sync job	Short TTL due to compliance sensitivity

6. JPA Repository Interfaces

```
public interface RuleVersionRepository         extends JpaRepository<RuleVersion,
Long> {      @Query(nativeQuery = true, value = """         SELECT rv.* FROM
schema_bre.bre_rule_version rv         JOIN schema_bre.bre_rule_definition rd ON
rd.id = rv.rule_definition_id         WHERE rv.status = 'ACTIVE'           AND
rd.product_type IN (:productType, 'ALL')           AND rd.applicability_stage
= :stage           AND rd.is_deleted = FALSE           AND (rv.effective_from IS
NULL OR rv.effective_from <= CURRENT_DATE)           AND (rv.effective_to   IS
NULL OR rv.effective_to   >= CURRENT_DATE)         ORDER BY rd.rule_category,
rv.version_number         """)     List<RuleVersion>
findActiveByProductTypeAndRuleGroup(         @Param("productType") String
productType,         @Param("stage") String stage);      Optional<RuleVersion>
findByRuleDefinitionIdAndStatus(         Long ruleDefinitionId, String status);
@Query("SELECT MAX(rv.versionNumber) FROM RuleVersion rv "          + "WHERE
rv.ruleDefinitionId = :ruleDefinitionId")     Optional<Integer>
findMaxVersionByRuleDefinitionId(         @Param("ruleDefinitionId") Long
ruleDefinitionId); }  public interface RuleExecutionLogRepository         extends
JpaRepository<RuleExecutionLog, Long> {      List<RuleExecutionLog>
findByLoanAppIdOrderByCreatedOnDesc(Long loanAppId);      @Query(nativeQuery =
true, value = """         SELECT rule_code,                COUNT(*) FILTER (WHERE
rule_status = 1) AS pass_count,                COUNT(*) FILTER (WHERE rule_status
= 2) AS deviation_count,                COUNT(*) FILTER (WHERE rule_status = 3) AS
fail_count,                COUNT(*) AS total_count         FROM
schema_bre.bre_rule_execution_log         WHERE rule_code = :ruleCode
```

```
AND created_on >= :fromDate         GROUP BY rule_code         """)
RuleExecutionSummary getRuleSummary(         @Param("ruleCode") String ruleCode,
@Param("fromDate") LocalDateTime fromDate); }
```

7. Error Handling Strategy

Scenario	HTTP / Outcome	Behaviour
Invalid rule condition DSL	400 Bad Request	Return validation errors per field; do not save rule
Missing mandatory input field	422 Unprocessable Entity	Return which fields are missing; evaluation aborted
Redis unavailable	Evaluation proceeds	Fall back to PostgreSQL; log warning; alert on repeated failures
Bureau API timeout	Rule marked PENDING_DATA	Retry with exponential backoff (3 attempts); then escalate
SpEL evaluation exception	Rule marked as ERROR	Log full stack trace; do not fail entire request; alert ops
Activate rule with no APPROVED status	409 Conflict	Reject with message: 'Rule must be in APPROVED status to activate'

8. Database Indexes & Performance

```
-- Rule lookup (hot path — hit on every evaluation) CREATE INDEX
idx_rule_version_active_product     ON schema_bre.bre_rule_version(status,
rule_definition_id)     WHERE status = 'ACTIVE';  -- Execution log queries CREATE
INDEX idx_exec_log_loan_app     ON schema_bre.bre_rule_execution_log(loan_app_id);
CREATE INDEX idx_exec_log_rule_code_date     ON
schema_bre.bre_rule_execution_log(rule_code, created_on DESC); CREATE INDEX
idx_exec_log_execution_group     ON
schema_bre.bre_rule_execution_log(execution_group_id);  -- Partitioning for
execution log (high volume) CREATE TABLE schema_bre.bre_rule_execution_log
PARTITION BY RANGE (created_on); -- Create monthly partitions: CREATE TABLE
bre_rule_execution_log_2026_06     PARTITION OF schema_bre.bre_rule_execution_log
FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
```

9. Supported Rule DSL Operators

Operator	Description	Example Value	Use Case
EQ	Equals	"A"	vrm_category = 'A'
NEQ	Not equals	"BLACKLISTED"	status != 'BLACKLISTED'
GT / GTE	Greater than / or equal	700	cibil_score >= 700

LT / LTE	Less than / or equal	0.5	foir <= 0.50
IN	Value in list	["A","B"]	vrm_category IN [A, B]
NOT_IN	Value not in list	["D","E"]	zone NOT IN [D, E]
BETWEEN	Range inclusive	[300000, 1000000]	loan_amount BETWEEN 3L-10L
IS_NULL	Field is null/missing	N/A	cibil_score IS_NULL (NTC customer)
EXPR	SpEL expression	"#total_emi / #income < 0.5"	Complex computed ratio checks